

Guía Didáctica: Promesas en JavaScript

Apartado 3: Estados de las promesas y manejo de errores

Explicación

Cada promesa en JavaScript puede estar en **tres estados** principales:

1. **Pending (pendiente):** La operación asíncrona aún no ha terminado.
2. **Fulfilled (cumplida):** La operación se completó correctamente y devolvió un valor.
3. **Rejected (rechazada):** La operación falló y produjo un error.

Manejo de errores:

- Se utiliza el método `catch()` para capturar cualquier fallo de la promesa.
- `finally()` se ejecuta siempre, independientemente de si la promesa fue cumplida o rechazada, útil para limpiar recursos o mostrar mensajes de finalización.

Ejemplo práctico 1: Estado pendiente y cumplimiento

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Estados de Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

<h1>Ejemplo 1: Estado Pendiente y Cumplido</h1>
<button id="btnEstado1" class="btn btn-primary">Iniciar
Promesa</button>
<div id="resultadoEstado1" class="mt-3"></div>

<script>
const btnEstado1 = document.getElementById('btnEstado1');
const resultadoEstado1 = document.getElementById('resultadoEstado1');

btnEstado1.addEventListener('click', () => {
  resultadoEstado1.innerHTML = `<div class="alert alert-
info">Promesa pendiente...</div>`;

  const promesa = new Promise((resolve, reject) => {
    setTimeout(() => resolve("Promesa cumplida!"), 2000);
  });
```

```

        promesa.then(msg => {
            resultadoEstado1.innerHTML = `<div class="alert alert-
success">${msg}</div>`;
        });
    });
</script>

</body>
</html>

```

Explicación:

- Inicialmente mostramos que la promesa está pendiente.
- Tras 2 segundos, se cumple (fulfilled) y se muestra el mensaje.

Ejemplo práctico 2: Manejo de error con catch

```

<h2>Ejemplo 2: Promesa Rechazada</h2>
<button id="btnEstado2" class="btn btn-warning">Iniciar
Promesa</button>
<div id="resultadoEstado2" class="mt-3"></div>

<script>
const btnEstado2 = document.getElementById('btnEstado2');
const resultadoEstado2 = document.getElementById('resultadoEstado2');

btnEstado2.addEventListener('click', () => {
    const promesa = new Promise((resolve, reject) => {
        setTimeout(() => {
            const exito = false;
            exito ? resolve("Todo bien!") : reject("Ocurrió un
error!");
        }, 2000);
    });

    promesa
        .then(msg => resultadoEstado2.innerHTML = `<div class="alert
alert-success">${msg}</div>`)
        .catch(err => resultadoEstado2.innerHTML = `<div class="alert
alert-danger">${err}</div>`);
    });
</script>

```

Explicación:

- reject() indica que la promesa falló.
- catch() permite capturar el error y manejarlo de forma segura.

Ejemplo práctico 3: Uso de finally

```

<h2>Ejemplo 3: finally</h2>
<button id="btnEstado3" class="btn btn-info">Iniciar Promesa</button>

```

```

<div id="resultadoEstado3" class="mt-3"></div>

<script>
const btnEstado3 = document.getElementById('btnEstado3');
const resultadoEstado3 = document.getElementById('resultadoEstado3');

btnEstado3.addEventListener('click', () => {
  const promesa = new Promise((resolve, reject) => {
    setTimeout(() => {
      const exito = Math.random() > 0.5;
      exito ? resolve("Promesa cumplida!") : reject("Promesa
fallida!");
    }, 2000);
  });

  promesa
    .then(msg => resultadoEstado3.innerHTML = `<div class="alert
alert-success">${msg}</div>`)
    .catch(err => resultadoEstado3.innerHTML = `<div class="alert
alert-danger">${err}</div>`)
    .finally(() => console.log("Promesa finalizada, éxito o
fallo."));
});
</script>

```

Explicación:

- `finally()` se ejecuta siempre, ideal para **limpiar o notificar** que la operación terminó.

Ejemplo práctico 4: Estados visuales en pantalla

```

<h2>Ejemplo 4: Mostrar estados en tiempo real</h2>
<button id="btnEstado4" class="btn btn-success">Iniciar
Promesa</button>
<div id="resultadoEstado4" class="mt-3"></div>

<script>
const btnEstado4 = document.getElementById('btnEstado4');
const resultadoEstado4 = document.getElementById('resultadoEstado4');

btnEstado4.addEventListener('click', () => {
  resultadoEstado4.innerHTML = `<div class="alert alert-
info">Estado: Pending...</div>`;

  const promesa = new Promise((resolve, reject) => {
    setTimeout(() => {
      const exito = Math.random() > 0.5;
      exito ? resolve("Estado: Fulfilled!") : reject("Estado:
Rejected!");
    }, 2000);
  });

  promesa
    .then(msg => resultadoEstado4.innerHTML = `<div class="alert
alert-success">${msg}</div>`)

```

```

        .catch(err => resultadoEstado4.innerHTML = `<div class="alert
alert-danger">${err}</div>`)
        .finally(() => resultadoEstado4.innerHTML += `<div
class="alert alert-secondary">Operación finalizada</div>`);
    });
</script>

```

Explicación:

- Se muestran los **tres estados de la promesa** de manera visual en pantalla: pending, fulfilled/rejected y finalización.

Ejemplo práctico 5: Manejo de errores en Fetch

```

<h2>Ejemplo 5: Fetch con catch y finally</h2>
<button id="btnFetchEstado" class="btn btn-secondary">Cargar
Datos</button>
<div id="resultadoFetchEstado" class="mt-3"></div>

<script>
const btnFetchEstado = document.getElementById('btnFetchEstado');
const resultadoFetchEstado =
document.getElementById('resultadoFetchEstado');

btnFetchEstado.addEventListener('click', () => {
    resultadoFetchEstado.innerHTML = `<div class="alert alert-
info">Cargando datos...</div>`;

    fetch('https://jsonplaceholder.typicode.com/usuarios_inexistentes')
        .then(response => {
            if(!response.ok) throw new Error("Error HTTP: " +
response.status);
            return response.json();
        })
        .then(data => {
            resultadoFetchEstado.innerHTML = `<div class="alert alert-
success">Datos cargados</div>`;
        })
        .catch(err => {
            resultadoFetchEstado.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
        })
        .finally(() => {
            console.log("Operación de fetch finalizada");
        });
    });
</script>

```

Explicación:

- `fetch()` devuelve una promesa.
- `catch()` **captura errores de red o de HTTP**.
- `finally()` permite realizar acciones finales, como ocultar un spinner de carga.

Ejercicios prácticos resueltos para alumnos

1. Crear una promesa que muestre “Pending...” y después de 3 segundos se cumpla con un mensaje de éxito.
2. Crear una promesa que se rechace si el número aleatorio es menor que 0.3 y mostrarlo con `catch()`.
3. Usar `finally()` para mostrar siempre un mensaje “Operación terminada” después de cualquier promesa.
4. Usar `fetch()` a un endpoint válido y capturar errores si la URL es incorrecta.

Ejercicios propuestos para alumnos

1. Crear una promesa que simule descargar un archivo y muestre el estado de descarga: pending → fulfilled/rejected → finalizado.
2. Usar `fetch()` para obtener posts y mostrar un mensaje de error si falla la petición.
3. Encadenar promesas y usar `finally()` para indicar que todas las operaciones terminaron.
4. Crear una promesa que devuelva datos de un formulario y capture errores si los campos están vacíos.